

PROJECT DOCUMENTATION

Script-Controlled ACL – Restrict Record Access Based on Field Value

TEAM DETAILS

Team ID : SWTID-2026-2771

Team Size : 5

Team Leader : ABINAYA K

Team member : ABINAYA G

Team member : AKALYA N

Team member : AMUTHA C

Team member : ANUPRIYA P

1. Introduction

Access Control Lists (ACLs) are used to control who can view, create, update, or delete records in an application. In some applications, access may need to depend on a particular field value stored in the record. This project demonstrates how a script-controlled ACL can restrict record access based on a field value.

2. Problem Statement

Traditional role-based access control may allow a user to access records even when the record belongs to another team or does not meet a required enrollment condition. The objective is to add a dynamic field-level condition so that access is granted only when the record value matches the required rule.

3. Objectives

- Implement a Script-Controlled ACL for record access.
- Restrict access based on a field value.
- Validate the condition dynamically when a record is accessed.
- Allow authorized records while denying unauthorized records.
- Improve data security and controlled access.

4. Scope

This project focuses on record-level access control using a scripted condition. It can be adapted for team enrollment, employee records, project records, approval records, or any table where access depends on a field value.

5. Technologies / Components

Component	Purpose
ACL	Controls access to records
Script / Condition	Checks the field value dynamically

Database Table	Stores enrollment/team records
User / Role	Identifies the person requesting access

6. Functional Requirement

The system must check the value of the selected field before allowing access to the record. For example, if the enrollment record contains a field named team_status, access can be allowed only when its value is Confirmed.

7. ACL Logic

Test Case	Field Value	Expected Result
TC-01	Confirmed	Access Allowed
TC-02	Pending	Access Denied
TC-03	Rejected	Access Denied

TC-04	Empty / Null	Access Denied
-------	--------------	---------------

The ACL evaluates the record at the time of access.

```
IF team_status = 'Confirmed' → Allow access
IF team_status ≠ 'Confirmed' → Deny access
```

8. Sample Script

```
// Script-Controlled ACL // Allow access only for
confirmed team enrollment records

answer = (current.team_status == 'Confirmed');
```

9. Implementation Steps

1. Create or identify the table containing team enrollment records.
2. Ensure the required field, such as team_status, is available on the record.
3. Create an ACL for the required operation, such as read access.
4. Select the appropriate table and operation.
5. Add the required role or role-based condition if applicable.
6. Add the script-controlled condition.
7. Save and test the ACL with different field values.

10. Test Cases

11. Expected Output

When a user attempts to access an enrollment record, the ACL checks the team_status field. Records with the value Confirmed are accessible to users who satisfy the ACL requirements. Records with other values are restricted.

12. Advantages

- Provides dynamic record-level security.
- Reduces unauthorized access to records.
- Uses field values to make access decisions.
- Can be customized for different business rules.
- Easy to test and maintain when the rule is clearly defined.

13. Limitations

The exact ACL behavior depends on the platform configuration, table permissions, roles, and other ACL rules. A scripted ACL should therefore be tested together with the complete access-control configuration.

14. Conclusion

The project demonstrates how a Script-Controlled ACL can be used to restrict record access based on a field value. By checking the enrollment status dynamically, the system can allow access only to records that meet the defined condition. This approach provides an additional layer of security and can be adapted to many real-world applications.

15. Future Enhancements

- Add multiple team-based conditions.
- Restrict access based on both user role and field value.
- Add audit logging for denied access attempts.
- Create automated test cases for ACL validation.
- Extend the logic to create, update, and delete operations.